

Applying Text-to-SQL in Process Mining: Leveraging Natural Language for Data Insights



Bruno Yui Yamate, Thais Rodrigues Neubauer, Marcelo Fantinato,
and Sarajane Marques Peres

Abstract Accessing a database using natural language has the potential to broaden information retrieval, making it accessible to users without SQL knowledge. This task, known as text-to-SQL, can also benefit the area of process mining, which provides tools to extract valuable insights from event logs. However, the text-to-SQL task in the process mining domain has not been fully explored. In this paper, we evaluate the text-to-SQL task using the *text₂SQL₄PM* dataset, a process mining domain-specific dataset built to serve as a benchmark for text-to-SQL implementations on process mining domain. We evaluated three large language models using different prompt strategies and representations. A detailed analysis of the results was conducted, providing insights for understanding the usability and feasibility of applying text-to-SQL on process mining domain.

Keywords Text-to-SQL · Process mining · Large language models

1 Introduction

The text-to-SQL task consists of giving a natural language utterance generates a corresponding SQL statement that can be executed on a database to answer the intention of that utterance [20]. Requesting information on a database through natural language has the potential to make information retrieval accessible to users without knowledge of SQL commands or syntax. Advancements in deep learning networks have allowed

B. Y. Yamate (✉) · T. R. Neubauer · M. Fantinato · S. M. Peres
School of Arts, Sciences and Humanities, University of São Paulo, São Paulo, SP, Brazil
e-mail: brunoyui@usp.br

T. R. Neubauer
e-mail: thais.neubauer@usp.br

M. Fantinato
e-mail: m.fantinato@usp.br

S. M. Peres
e-mail: sarajane@usp.br

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
A. Rocha et al. (eds.), *Emerging Trends in Information Systems and Technologies*,
Lecture Notes in Networks and Systems 1582,
https://doi.org/10.1007/978-3-032-01130-5_16

191

the text-to-SQL task to be solved by applying an encoder-decoder architecture [19, 22] and PLM¹ models [7, 12, 18] to generate SQL statements from a natural language utterance. More recently, LLM² based models emerged as novel implementations for the text-to-SQL task with diverse prompt engineering strategies [4, 9]. Regardless of the implementation, three benchmarks [5, 21, 22] are commonly used for tuning or evaluating such models.

Although the aforementioned benchmark datasets span multiple domains ranging from cars, finance, pets, sports, flights, and more, it is not clear how the model's implementation affects usability and feasibility on a more specialized domains with specific requirements for information retrieval. The process mining domain is one of these cases. Process mining is an interdisciplinary field in which data of interest are stored in a structure known as event log, which records events of a business process execution [15]. From event logs, users need to extract valuable insights, either through specialized algorithms or information retrieval methods. Although event logs are not necessarily stored in a relational database, they can be transformed for SQL-based information retrieval to extract valuable insights.

In process mining literature, the use of LLMs to generate SQL statements has been explored through prompts incorporating business process information and process mining instructions [1]. While this approach aimed to assess LLMs' feasibility for information retrieval, the development of more advanced prompts hinders a clear evaluation of their capabilities and limitations in this domain. In this paper, we evaluate basic prompt strategies on three large language models: GPT-3.5 Turbo [8], Gemini-1.0 Pro [13] and LLama3-8B Instruct [14] on *text₂SQL₄PM* benchmark dataset. Therefore, the contributions of this paper are as follows:

1. Understand the usability and feasibility of text-to-SQL implementations applied to the process mining domain through an evaluation with 0-shot and 1-shot prompt strategies on a process mining domain-specific benchmark dataset, named *text₂SQL₄PM*, following the work done on DAIL-SQL [4].
2. A categorization inspired by NL2SQL360 [6] to assist in analyzing the models results, focusing on SQL complexity and SQL pattern templates.

This paper is organized as follows: Sect. 2 provides an overview of basic concepts of text-to-SQL and process mining. Section 3 discusses related work. Section 4 outlines the method, including the dataset, implementation details, and evaluation strategies. Section 5 presents the results and a detailed analysis. Finally, Sect. 6 concludes with a summary of findings and future directions.

¹ PLM: tunable pre-trained language models.

² LLM: large language models.

2 Preliminaries

2.1 *Text-to-SQL Benchmarks and Implementations*

In the text-to-SQL task, a natural language utterance is taken as input and a SQL statement is generated as output, which can then be executed on a database. There are three well-known text-to-SQL benchmarks for evaluating implementations: WikiSQL [22], Spider [21] and Bird [5]. WikiSQL is the simplest benchmark according to schema structure, with only a single table and simple SQL statements. The Spider benchmark is more robust than WikiSQL, consisting of SQL statements with complex clauses and a schema structure with tables containing relationships between them. The Bird benchmark contains a more robust dataset than Spider, with pairs of utterances that need domain knowledge to be answered, and SQL statements with complex clauses and function operators.

For a long time, the majority of text-to-SQL implementations that achieved reasonable results on the WikiSQL and Spider benchmarks relied on supervised fine-tuned encoder-decoder neural network architectures. Initially, the utterance input was encoded with a bidirectional LSTM on Seq2SQL [3] and SQLNet [19], and the decoder, conditioned on the representation produced by the encoder, generates SQL statements on Seq2SQL [3] or filled the slots for each SQL clause on SQLNet [19]. For Spider benchmark, which also requires database schema representations as input, a graph neural network was used to encode the utterance and the schema representation on GNN [2] and RATSQ [18], and the decoder, conditioned on that representation, generates SQL statements. Other approaches used pre-trained language models like GAP [12] and T5 [10] to produce contextualized input representations and, using an encoder-decoder architecture like RAT-SQL [18] or RESDSQL [12], generates SQL statements. With advancements in large language models achieving state-of-the-art results on the Spider benchmark, the Bird benchmark was created. For Bird benchmark, text-to-SQL implementations use large-language models with different strategies for prompt inputs to generate SQL statements, such as DIN-SQL [9] and DAIL-SQL [4].

2.2 *Process Mining and Event Logs*

Process mining is positioned at the intersection of two fields of knowledge: data science and process science [15]. The primary source of information for process mining is event logs, which serve as the foundation for applying analysis strategies to identify workflow bottlenecks, anticipate performance issues, assess conformance and compliance, recommend countermeasures, and optimize business processes [17]. Although the common standard for storing the event logs is XES (eXtensible Event Stream) format, it can be converted into a non-normalized database relation, which query languages like Structure Query Language (SQL) can be used to extract valuable insights.

An event log records a sequence of *events* related to execution of activities on a business process under analysis. This business process can be executed multiple times, with each execution referred to as a *process instance*, which is commonly referred to as *case*. Each case is unique and can be identified by a *caseID*. Traditionally, each event is related to only one case, and a case refers to a unique sequence of events. In addition to information about the case to which each event is linked, other attributes can be recorded for each event, such as the activity performed (an essential attribute), the resource (person) involved in its execution, and the associated cost. A mandatory requirement is that the order of events within a case reflects the order of their occurrences over time, which is often recorded in the event log by the *timestamp* attribute [15].

3 Related Work

The adoption of large language models revolutionized the text-to-SQL task. Through 0-shot or few-shot prompts and different strategies for input prompts, LLM models based on GPT-3.5 Turbo [8], Gemini-1.0 Pro [13] and Llama3-8B Instruct [14] can generate SQL statements given a natural language utterance. DAIL-SQL [4] made comparisons through a series of experiments, in which different input representations were given with 0-shot prompts for large language models. Additionally, another comparison using few-shot prompts for the in-context learning approach was conducted with different examples selection and organization. Example selection refers to determining which strategy (ex.: random, question similarity, etc.) to select examples from a sample dataset to provide for context prompt. Organization refers to which information (ex.: utterance, utterance + SQL statement, etc.) from examples selected to provide for context prompt. The experiments used Spider [21] and Bird [5] benchmarks.

Along with the adoption of LLM-base models to generate SQL statements, it's important to explore methods for analyzing the results of these models beyond the commonly used metrics. NL2SQL360 [6] provides a testbed for the evaluation of text-to-SQL implementations. Through NL2SQL360 a series of experiments were conducted comparing various PLM models and LLM models on four perspectives: domain-specific databases (ex.: sports, finance, etc.), SQL complexity using Spider and Bird criteria, SQL characteristics (ex.: logical operators, subquery, JOIN operator, etc.) and natural language variance. The experiments were conducted with the Spider [21] and Bird [5] benchmarks.

In this paper, we evaluate strategies similar to DAIL-SQL [4] for processing prompt inputs on the *text₂SQL₄PM* dataset. We also provide a comprehensive analysis of the results, using SQL pattern templates and SQL complexities, similar to the approach used in NL2SQL360 [6].

4 Method

This section presents the method used to evaluate the Text-to-SQL task in the process mining domain. It introduces the process mining dataset used as benchmark, followed by the models and prompt strategies employed to generate SQL statements, and concludes with the evaluation strategies.

Process mining dataset: To evaluate the text-to-SQL task in the process mining domain, a bilingual dataset entitled *text₂SQL₄PM* was used³. The dataset comprises 1655 quadruples (NL-PT utterance, NL-EN utterance, SQL statement—PT, SQL statement—EN), consisting of 205 base utterances and 1450 paraphrases. Initially, the dataset was developed as an exercise for Data Mining and Process Mining classrooms, and it later underwent a curation process conducted by process mining experts. A professional translator was involved to ensure the correctness of translations between languages. An event log related to authorization requests for reimbursement of academic travel expenses [16] was used during the curation process to standardize the dataset's business process focus. Besides the set of quadruples, the dataset owners also provide classifications for the quadruples, which they refer to as 'qualifiers'. Thus, each quadruple is annotated with ten qualifiers, which are perspective categories⁴ used to help organize the dataset and facilitate the evaluation of the results.

Models and prompt strategies: We evaluated three large language models: GPT-3.5 Turbo [8], Gemini-1.0 Pro [13] and LLama3-8B Instruct [14]. The OpenAI representation (OPR) and Code representation (CR), as used in the DAIL-SQL [4], were chosen as strategies for prompts inputs in large language models to obtain the SQL statements in both languages, Portuguese and English. The prompts were designed using a 0-shot approach, in which only the natural language utterance and database schema are provided as input, and a 1-shot approach, in which similar examples are included as context for prompt input.

For the 1-shot approach, the examples were provided with full information, so the prompt context contains examples with the database schema information, the utterance and the SQL statement. The selection of examples for the input prompt was made based on the similarity between the example utterances and the target utterance, using the cosine distance measure. All the example utterances and the target utterances were encoded to obtain the embeddings using Sentence-BERT [11], and a cosine similarity was calculated between each of the target embeddings and all the example embeddings, selecting the example with highest result (the most semantically similar). For the selection of examples by similarity, we excluded the paraphrases created for that specific target utterance.

³ Available at <https://github.com/pm-usp/text-2-sql>.

⁴ Natural language perspective (wh-question and base-paraphrase categorization); process mining perspective (event-case level, value-domain-generic, perspective-descriptive statistics-conformance categorization); SQL perspective (complexity and SQL operators).

Table 1 Details and descriptions of grammar symbols of SQL pattern templates—concept: a attribute that have meaning on process mining domain (resource, activity, event, cost, timestamp, case).

Symbol	Description
+	One or more
*	Zero or one
Sc	A concept of interest on SELECT clause
Ac	An aggregation on a concept of interest on SELECT clause
Acc	An aggregation on a concept of interest grouped by another concept on SELECT clause
Fc	A filter on a concept presented on WHERE clause or if appears after Gc symbol represent a filter on HAVING clause
FSc	A filter on a concept with SUBSELECT presented on WHERE clause or HAVING clause if appears after Gc symbol
Gc	A concept grouped by GROUP clause
SIE	A combination of set operations with INTERSECT or EXCEPT clauses between two queries
SU	A combination of a union operation with UNION clause between two queries
RS	A SUBSELECT result on a JOIN or FROM clause
SUE	A combination of set operations with EXCEPT clause or a union combination with UNION clause between two queries
WS	A SQL statement that contains WITH clause

Evaluations strategies: The two well-known text-to-SQL metrics were adopted to evaluate the models, the *Exact Match Accuracy* (EM) and the *Execution Accuracy* (EX) [21]. The EM metric is a structural metric that evaluates if the target SQL statement is exactly equal to the SQL statement produced by the model. The EM metric does not evaluate values in the condition clauses, which can lead to false positives results; therefore, the EX metric can be used in tandem. The EX metric is a run metric that evaluates if the results by the execution of target SQL statement are equal to the results by the execution of SQL statement produced by the model. For the EX metric, SQL statements with different structures that produce the same results are considered correct. A downside of this metric is that SQL statements with semantically different that produce the same results are considered correct (ex.: both SQL statements do not produce any result).

To guide a more detailed analysis of the results, SQL pattern templates were identified, and a grammar-like representation was created to identify these patterns. All SQL pattern templates are presented on github repository⁵ and details of each symbol of this grammar like is explained on Table 1. Due to space constraints, in this paper we present only the results of text-to-SQL in the context of the English

⁵ Available at <https://github.com/pm-usp/text-to-SQL-pm>.

Table 2 Results of EX metric for 0-shot and 1-shot prompt strategies using OpenAI and Code representations, OPR and CR respectively

	GPT-3.5 Turbo		Gemini-1.0 Pro		Llama3-8B Instruct	
	OPR	CR	OPR	CR	OPR	CR
0-shot	47.61	46.47	39.70	37.52	36.13	39.09
1-shot	47.85	54.56	38.67	47.73	39.40	47.79

language, utilizing the *Execution Accuracy* (EX) evaluation metric. The analyses of these results take into account the complexities of SQL statements and SQL pattern templates. The complete results using *Exact Match Accuracy* (EM) and *Execution Accuracy* (EX), as well as the SQL statements generated by the LLMs themselves, for both English and Portuguese languages are provided on github repository (see Footnote 5).

5 Results and Analysis

EX metrics results: The EX metric results using OpenAI representation and Code representation with 0-shot and 1-shot are shown in Table 2. For 0-shot, the OpenAI representation shows better results for GPT-3.5 Turbo and Gemini-1.0 Pro, while the Code representation shows better results for LLama3-8B Instruct, with the best results occurring on GPT-3.5 Turbo using OpenAI representation, with 47.61%. For 1-shot, the Code representation shows better results for all models, with the best results occurring on GPT-3.5 Turbo using Code representation, with 54.56%, resulting in an accuracy increase of 6.95% compared to the 0-shot best result on GPT-3.5 Turbo using OpenAI representation.

SQL pattern templates results and analysis: An analysis was conducted to identify the main challenges in variations of SQL statements in *text₂SQL₄PM* dataset. Challenges encountered are related to: DS) domain-specific terms; IVFR) interpretation of values for filtering results; AMS) additional or missing attributes in the SELECT clause; PMC) understanding and interpretation of process mining concept—*case*; IEA) interchangeable use of the terms *activity* and *event* in process mining, which complicates the selection of the appropriate term in various contexts. Examples of each challenge are detailed on Table 3 and referenced when appropriate. Results, along with complexity classes (easy, medium, hard, extra hard and no_hardness⁶) [21] are presented for each SQL pattern template.

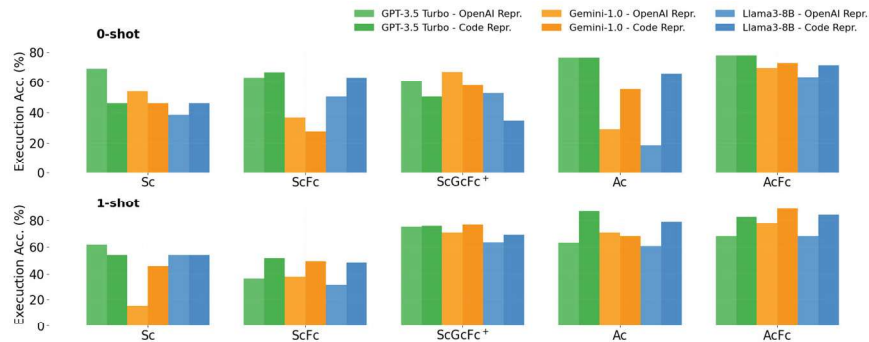
For the **easy class** in the SQL complexity qualifier, five SQL pattern templates were found: Sc, ScFc, ScGcFc⁺, Ac, and AcFc. The results illustrated in Fig. 1 shows

⁶ no_hardness: a class which a gold SQL statement could not be classified due limitations of the scripts of Spider benchmark (ex.: use of NULL and IS NULL operators, use of function STRFTIME, etc.).

Table 3 Examples of each challenge

	Utterance	Challenge
DS	Which declarations were rejected by a pre-approver other than Douglas ?	Understanding declarations term refers to a case, a declaration rejected by a pre-approver corresponds to the value 'Declaration rejected by pre-approver', and Douglas is identified as a resource
IVFR	How many times were ' permit ' activities performed in each case?	It is necessary to recognize that there are many types of permitted activities (ex.: permit submitted by employee)
AMS	Which resource was most active in the 'declaration approved by administration' activity?	Some of the resulting SQL statements included a count alongside the entity 'resource'
PMC	What is the average duration of each activity?	Knowledge of the case, including the sequence of activities and the duration of each activity, must be calculated for each case before computing the average
IEA	How many activities were associated with each resource?	Confusion regarding the term 'activities', which could lead to ambiguity

DS domain-specific terms; *IVFR* interpretation of values for filtering results; *AMS* additional or missing attributes in SELECT clause; *PMC* process mining case concept; *IEA* interchangeable use of events and activity

**Fig. 1** EX metric results with 0-shot and 1-shot prompt for SQL pattern templates of easy class in the SQL complexity qualifier

that even for relatively simple SQL statements like Sc and ScFc, the models did not achieve optimal performance. In some cases, performance decreased using 1-shot prompt strategy, highlighting challenges in interpreting natural language utterance terms for projection and filtering results (challenge IVFR cf. Table 3). For ScGcFc+, Ac and AcFc, the challenges of term interpretation also occur, highlighting the errors in domain-specific terms without explicit connections between the items of database schema and the utterance terms (challenge DS cf. Table 3). Problems also occur with additional attributes not needed in SELECT clause (challenge AMS cf. Table 3).

For the **medium class** in the SQL complexity qualifier, seven SQL pattern templates were found: Sc^+ , $ScGc$, Sc^+Fc^+ , Sc^+GcFc^+ , Acc^+Gc , $Ac(c^+)^*Fc^+(Gc)^*$, and $(Ac)^+c(Fc)^*GcFc$. The results illustrated in Fig. 2 shows that Sc^+ , $ScGc$, Sc^+Fc^+ and Sc^+GcFc^+ struggle with domain-specific terms without explicit connections between the items of the database schema and the utterance terms (challenge DS cf. Table 3) and problems with interpreting values for filtering results (challenge IVFR cf. Table 3). The $ScGc$ 0% accuracy of some models is due to the low number of samples in that category. For Acc^+Gc , $Ac(c^+)^*Fc^+(Gc)^*$ and $(Ac)^+c(Fc)^*GcFc$, the models encounter problems with domain-specific terms (challenge DS cf. Table 3), interpretation of utterance terms for aggregation (challenge IVFR cf. Table 3) and interchangeable use of event and activity terms (challenge IEA cf. Table 3).

For the **hard class** in the SQL complexity qualifier, seven SQL pattern templates were found: $Sc^+Fc^+(Gc)^*$, $ScFSc$, $ScFcGcFc$, $ScGcFSc$, $AcFSc$, $AccFc^+Gc$, and SIE . The results illustrated in Fig. 3 shows that $Sc^+Fc^+(Gc)^*$ and $ScFcGcFc$ present problems with domain-specific terms (challenge DS cf. Table 3). $ScFSc$ and $ScGcFSc$, which involve a more complex SQL statement with SUBSELECT, present

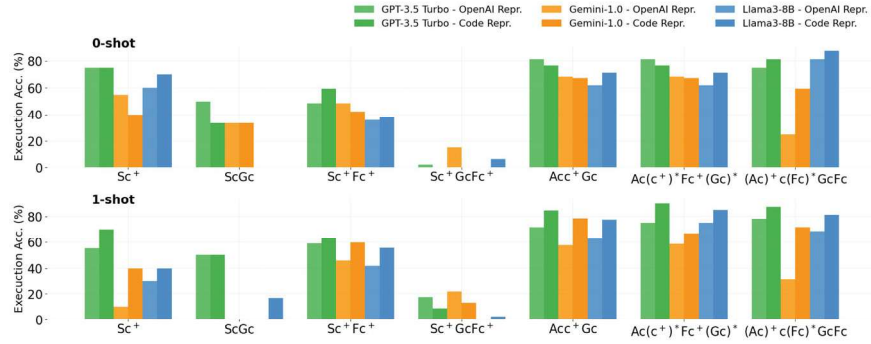


Fig. 2 EX metric results with 0-shot and 1-shot prompt for SQL pattern templates of medium class in the SQL complexity qualifier

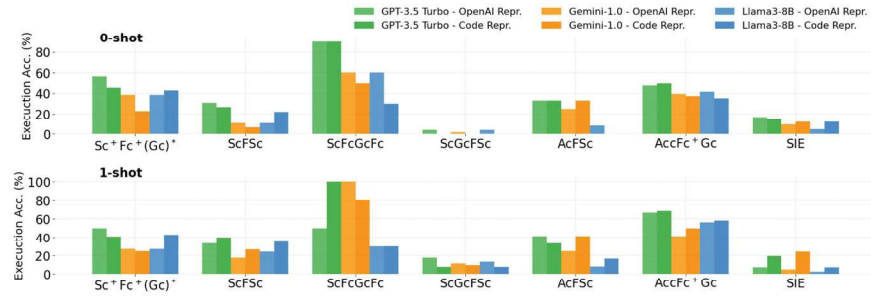


Fig. 3 EX metric results with 0-shot and 1-shot prompt for SQL pattern templates of hard class in the SQL complexity qualifier

challenges for the subtle interpretation of utterances that involve reasoning with plural terms (challenge IVFR cf. Table 3) (ex.: Which resources participated in the most events) meaning that there are more than one resource. AcFSc and AccFc⁺Gc present problems with domain-specific terms (challenge DS cf. Table 3) and interpretation of values for filtering results (challenge IVFR cf. Table 3). For SIE, the models struggle with the use of INTERSECT and EXCEPT SQL operators, seeming to avoid them in SQL generation.

For the **extra hard class** in the SQL complexity qualifier, seven SQL pattern templates were found: ScFc⁺(Gc)*, ScFSc(Fc)*(Gc)*, Sc(Fc)*GcFSc, Acc⁺Fc⁺Gc⁺, AcFSc, AccGc(FSc)*, and SU. The results illustrated in Fig. 4 shows that ScFc⁺(Gc)* and Acc⁺Fc⁺Gc⁺ present problems with domain-specific terms (challenge DS cf. Table 3), interpretation of values for filtering results (challenge IVFR cf. Table 3) and additional or missing attributes in SELECT clause (challenge AMS cf. Table 3). SQL pattern templates AcFSc and ScFSc(Fc)*(Gc)* resulted in 0% accuracy for all models using 0-shot, and Sc(Fc)*GcFSc resulted in practically 0% accuracy for all models except Llama3-8B Instruct OpenAI representation, indicate challenges for SQL that involves SUBSELECT (1-shot strategy could improve such results). That challenge occurs due to the implicit knowledge of *case* concept (challenge PMC cf. Table 3), necessary to properly SQL statement generation that not occurs on utterances for pattern AccGc(FSc)*. For SU, which involves UNION operation, the models also appear to avoid using them in SQL statement generation.

For the **‘no_hardness’ class** in the SQL complexity qualifier, ten SQL pattern templates were found: Sc⁺Fc⁺(Gc)*, Sc⁺Gc(Fc)*, Sc⁺(Fc)*FSc(Gc)*, Sc⁺RS(Fc)*, Ac(c)*Fc⁺(Gc)*, AccFScGc, AccGc, Ac(c)*RS(Fc)*(Gc)*, S(UE)⁺ and WS. The results illustrated in Fig. 5 shows 0% accuracy on five SQL pattern templates on a 0-shot prompt and lower results in the others templates, except on AccGc. Besides the previously mentioned issues in previous complexity classes and also present in the ‘no hardness’ class, one challenge becomes evident in the ‘no hardness’ class across all SQL pattern templates: The utterances in the category that require a fundamental understanding of the process mining *case* concept (challenge PMC cf. Table 3). Such

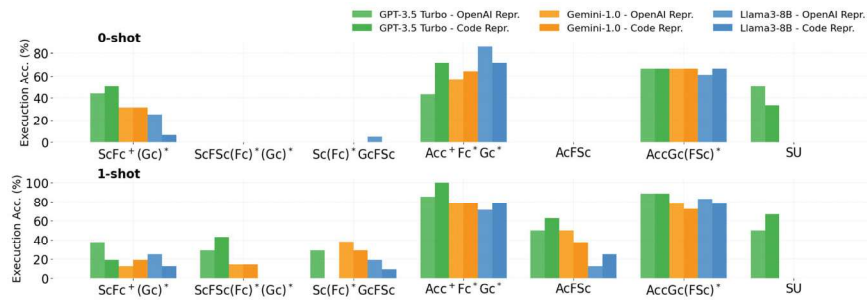


Fig. 4 EX metric results with 0-shot and 1-shot prompt for SQL pattern templates of extra hard class in the SQL complexity qualifier

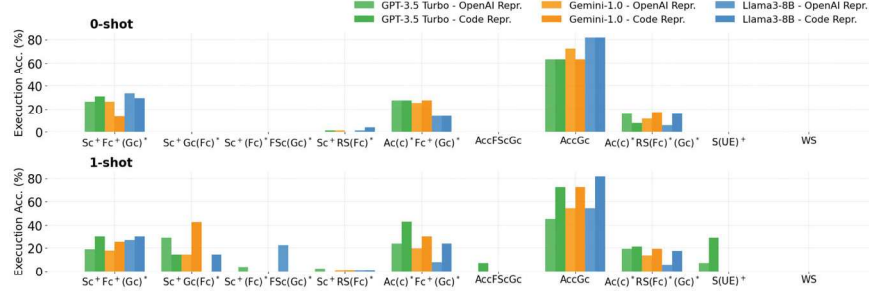


Fig. 5 EX metric results with 0-shot and 1-shot prompt for SQL pattern templates of no_hardness class

knowledge implies that models must understand that a case has a start, an end, and events in between that occur sequentially, ordered by their occurrence time. This notion, implicit in the utterances, consequently implies that complex SQL statements must be generated in response, which seems to improve in some cases with the 1-shot prompt strategy.

While basic prompting strategies allow for a more direct evaluation of each LLM's capabilities, the results in Table 2 indicate that these strategies alone are insufficient to create a viable text-to-SQL implementation for the process mining domain. Additionally, as shown in Fig. 5, SQL statements categorized as 'no_hardness'—those with greater potential usability in this domain—yielded the poorest results. Therefore, developing a systemic solution requires advancing towards more detailed and informative prompts.

6 Conclusions

In this paper, we presented an approach to understand the usability and feasibility of text-to-SQL task using LLMs applied on process mining domain. Through *text₂SQL₄PM*, a process mining dataset created with pairs of utterances and SQL statements, we evaluated three LLM-based models (GPT-3.5 Turbo, Gemini-1.0 Pro and Llama3-8B Instruct) using OpenAI and Code representations with 0-shot and 1-shot prompt strategies. We categorized the SQL statements presented in the *text₂SQL₄PM* dataset by SQL pattern templates and complexity, allowing for a more detailed and comprehensive analysis, alongside with a well-known metric (EX) used to evaluate text-to-SQL implementations. In our experiment, the best results occurred on GPT-3.5 Turbo, with 47.61% for the 0-shot prompt using OpenAI representation and 54.56% for the 1-shot prompt using Code representation. A limitation of our experiment is the use of the simplest models within their respective families, selected for their affordability. Models with more parameters may yield different results, though these are likely to be correlated. Given the potential of large language

models, we hypothesize that the primary challenges identified in our analysis can be addressed by developing more advanced prompt engineering strategies, such as RAG, tailored to process mining concepts. Ideally, these strategies should be integrated into a reasoning method capable of generalizing across various business process domains. Testing this hypothesis will be a key focus of the next steps in this research line.

References

1. Berti, A., Schuster, D., van der Aalst, W.M.P.: Abstractions, scenarios, and prompt definitions for process mining with LLMs: a case study. In: Weerdt, J.D., Pufahl, L. (eds.) *Business Process Management Workshops*, pp. 427–439. Springer, Cham (2024)
2. Bogin, B., Gardner, M., Berant, J.: Representing schema structure with graph neural networks for text-to-SQL parsing. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 4560–4565. Association for Computational Linguistics, Florence, Italy (2019)
3. Dong, L., Lapata, M.: Language to Logical Form with Neural Attention. *CoRR* **abs/1601.01280** (2016)
4. Gao, D., Wang, H., Li, Y., Sun, X., Qian, Y., Ding, B., Zhou, J.: Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *CoRR* **abs/2308.15363** (2023)
5. Li, J., Hui, B., Qu, G., Li, B., Yang, J., Li, B., Wang, B., Qin, B., Cao, R., Geng, R., Huo, N., Ma, C., Chang, K.C.C., Huang, F., Cheng, R., Li, Y.: Can LLM Already Serve as A Database Interface? A BIG Bench for Large-Scale Database Grounded Text-to-SQLs (2023)
6. Li, B., Luo, Y., Chai, C., Li, G., Tang, N.: The dawn of natural language to SQL: are we fully ready? In: *Proceedings of the VLDB Endowment*, vol. 17, pp. 3318–3331 (2024)
7. Li, H., Zhang, J., Li, C., Chen, H.: RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL (2023)
8. Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P.F., Leike, J., Lowe, R.: Training language models to follow instructions with human feedback. In: Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., Oh, A. (eds.) *Advances in Neural Information Processing Systems*, vol. 35, pp. 27730–27744. Curran Associates, Inc. (2022)
9. Pourreza, M., Rafiei, D.: DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction (2023)
10. Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., Liu, P.J.: Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.* **21**(1) (2020)
11. Reimers, N., Gurevych, I.: Sentence-BERT: Sentence embeddings using Siamese BERT-Networks. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics (2019)
12. Shi, P., Ng, P., Wang, Z., Zhu, H., Li, A.H., Wang, J., Santos, C.N., Xiang, B.: Learning contextual representations for semantic parsing with generation-augmented pre-training. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35(15), pp. 13806–13814 (2021)
13. Team, G.: Gemini: A Family of Highly Capable Multimodal Models. *arXiv* **2312.11805** (2024)
14. Team, L.: The Llama 3 Herd of Models. *arXiv* **abs/2407.21783** (2024)
15. van der Aalst, W.M.P.: *Process Mining: Data Science in Action*, 2nd edn. Springer, Heidelberg (2016)
16. van Dongen, B.: BPI Challenge 2020: Request For Payment (2020). https://data.4tu.nl/articles/dataset/BPI_Challenge_2020_Request_For_Payment/12706886/1

17. van der Aalst, W.M.P., Carmona, J. (eds.): *Process Mining Handbook*. Lecture Notes in Business Information Processing, vol. 448. Springer (2022)
18. Wang, B., Shin, R., Liu, X., Polozov, O., Richardson, M.: RAT-SQL: relation-aware schema encoding and linking for text-to-SQL parsers. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp 7567–7578. Association for Computational Linguistics, Online (2020)
19. Xu, X., Liu, C., Song, D.: SQLNet: Generating Structured Queries From Natural Language Without Reinforcement Learning. *CoRR* **abs/1711.04436** (2017)
20. Yu, T., Li, Z., Zhang, Z., Zhang, R., Radev, D.: TypeSQL: Knowledge-based type-aware neural text-to-SQL generation. In: *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Vol. 2 (Short Papers)*, pp. 588–594. Association for Computational Linguistics, New Orleans, Louisiana (2018)
21. Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., Radev, D.: Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 3911–3921. Association for Computational Linguistics, Brussels, Belgium (2018)
22. Zhong, V., Xiong, C., Socher, R.: Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning. *CoRR* **abs/1709.00103** (2017)